

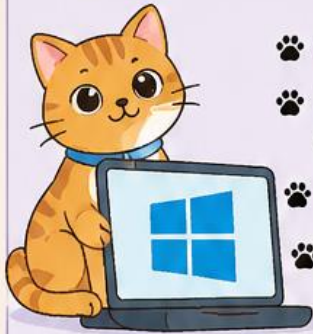


1. AUTENTICACIÓN SQL Y WINDOWS: DIFERENCIAS Y PRÁCTICAS SEGURAS

La autenticación es el proceso mediante el cual SQL Server verifica la identidad del usuario antes de permitir el acceso a la base de datos.



AUTENTICACIÓN WINDOWS



- Utiliza las cuentas de Windows.
- No requiere una contraseña adicional para SQL Server.
- Se integra con Active Directory.
- Es el método más seguro.

VENTAJAS

- ✓ Mayor seguridad
- ✓ Administración centralizada
- ✓ Soporta políticas de contraseña
- ✓ Inicio de sesión único (SSO)

AUTENTICACIÓN SQL SERVER



- Utiliza usuarios creados dentro de SQL Server.
- Cada usuario posee su propio nombre y contraseña.
- Es útil cuando no existe un dominio Windows.

EJEMPLO

Usuario: adminSQL

Contraseña: *****



COMPARACIÓN

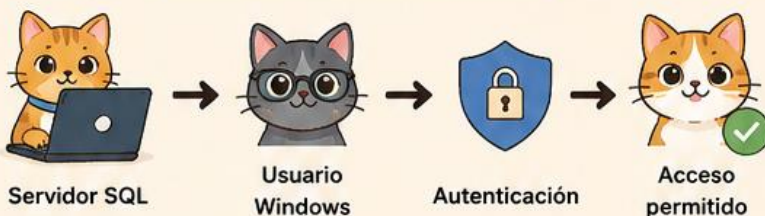
WINDOWS	SQL SERVER
 Más segura	 Menos segura
 Usa Active Directory	 Usuarios propios
 Soporta políticas de Windows	 Depende de SQL Server
 Recomendada	 Solo cuando sea necesario

-BUENAS PRÁCTICAS -

- ✓ Utilizar preferentemente Windows Authentication.
- ✓ Crear contraseñas fuertes.
- ✓ Deshabilitar cuentas que no se utilicen.
- ✓ Evitar usar la cuenta sa.
- ✓ Activar bloqueo por intentos fallidos.
- ✓ Aplicar el principio de mínimo privilegio.



EJEMPLO DE FLUJO



DATO IMPORTANTE



Microsoft recomienda utilizar Windows Authentication siempre que sea posible.





2. CUENTAS DE SERVICIO Y CONFIGURACIÓN DEL SERVIDOR 🐾

Las cuentas de servicio son las cuentas de Windows que utiliza SQL Server para ejecutar sus servicios. Una buena configuración mejora la seguridad, estabilidad y rendimiento del servidor.



¿QUÉ SERVICIOS UTILIZAN CUENTAS DE SERVICIO?

- SQL Server Database Engine ⚙️
- SQL Server Agent 🕒
- SQL Server Browser 🔍



🐾 TIPOS DE CUENTAS DE SERVICIO 🐾

1. LOCAL SYSTEM



- 🐾 Tiene muchísimos privilegios en el sistema.
- 🐾 No es recomendable para ambientes de producción.

2. NETWORK SERVICE



- 🐾 Tiene menos privilegios.
- 🐾 Acceso limitado a la red.

3. CUENTA DE DOMINIO



Ejemplo:
DOMINIO\SQLService

Permite:

- 🐾 RespalDOS en red
- 🐾 Acceso entre servidores
- 🐾 Autenticación integrada



Es la opción más usada en empresas por su seguridad y flexibilidad.

CONFIGURACIÓN IMPORTANTE DEL SERVIDOR 🐾



- ✓ Configurar memoria máxima.
- ✓ Configurar puertos TCP/IP.
- ✓ Habilitar protocolos necesarios.
- ✓ Configurar copias de seguridad.
- ✓ Mantener el servidor actualizado.



🐾 BUENAS PRÁCTICAS 🐾

- 👤 Usar cuentas dedicadas para cada servicio.
- 🛡️ No ejecutar SQL Server como Administrador.
- 🔒 Limitar los permisos al mínimo necesario.
- 🔑 Cambiar contraseñas periódicamente.
- 🔍 Monitorear los servicios regularmente.



BENEFICIOS DE UNA BUENA CONFIGURACIÓN 🐾



- ✓ Mayor estabilidad del servidor.
- 🔒 Mayor seguridad de la información.
- 📊 Mejor rendimiento en las operaciones.
- ✓ Menor riesgo de ataques o fallos.



¡RECUERDA! 🐾

Una cuenta de servicio bien configurada es la base para un SQL Server seguro, estable y confiable.



🐾 ¿CÓMO TRABAJA? 🐾



Cuenta de Servicio



Servicios de SQL Server



Ejecuta y protege el servidor

La cuenta de servicio es como el cuidador del servidor: si está bien elegida y configurada, todo funciona seguro y sin problemas.



3. CREACIÓN DE ROLES

FIJOS Y PERSONALIZADOS

Un rol es un conjunto de permisos que se asigna a uno o varios usuarios. Facilita la administración y mejora la seguridad de la base de datos.

¿QUÉ ES UN ROL?

Un rol agrupa permisos para asignarlos fácilmente a los usuarios en lugar de hacerlo uno por uno.

VENTAJAS

- ✓ Administración sencilla.
- ✓ Mayor organización.
- ✓ Menor riesgo de errores.
- ✓ Facilita auditorías.

ROLES FIJOS

Son creados automáticamente por SQL Server.

- sysadmin**
Control total del servidor.
- db_owner**
Dueño de la base de datos.
- db_datareader**
Permite leer los datos.
- db_datawriter**
Permite modificar los datos.
- securityadmin**
Administra permisos y logins.

ROLES PERSONALIZADOS

Son creados por el administrador según las necesidades.

EJEMPLO



Rol: Secretaria

Permisos:

- ✓ Consultar estudiantes
- ✓ Registrar expedientes
- ✗ Eliminar registros

Se pueden crear roles para cada área o función de la institución.

¿CÓMO FUNCIONA?



BUENAS PRÁCTICAS

- ✓ Aplicar el principio de mínimo privilegio.
- ✓ Evitar usar el rol sysadmin.
- ✓ Agrupar usuarios por funciones.
- ✓ Revisar y actualizar roles periódicamente.
- ✓ Documentar los roles y permisos.

EJEMPLO EN LA VIDA REAL

Así como en una madriguera cada conejo tiene un rol diferente, en una base de datos cada usuario tiene permisos según su función.



EJEMPLO DE CREACIÓN DE ROL

Código de ejemplo para crear un rol personalizado y asignarle permisos:

```
-- Crear un rol personalizado
CREATE ROLE Secretaria;

-- Asignar permisos al rol
GRANT SELECT, INSERT
ON Estudiantes
TO Secretaria;

-- Agregar usuarios al rol
EXEC sp_addrolemember 'Secretaria', 'usuario1';
EXEC sp_addrolemember 'Secretaria', 'usuario2';
```

¡Los roles hacen todo más fácil y seguro!

Los roles permiten mantener el orden en la base de datos, asignando a cada usuario solo lo que necesita para realizar su trabajo.

4. CONTROL DE ACCESO MEDIANTE GRANT, DENY Y REVOKE

Son instrucciones SQL que permiten administrar permisos sobre objetos de la base de datos, controlando qué acciones puede realizar cada usuario.

GRANT: OTORGA PERMISOS

Permite autorizar a un usuario o rol para realizar una acción específica.



Se le da permiso para entrar y realizar la acción.

EJEMPLO

GRANT SELECT
ON Estudiantes
TO Tutor;

→ Permite al usuario "Tutor" consultar la tabla Estudiantes.

DENY: NIEGA PERMISOS

Niega explícitamente un permiso, incluso si el usuario lo tiene por herencia.



No tiene permiso para realizar esta acción.

EJEMPLO

DENY DELETE
ON Estudiantes
TO Tutor;

→ Niega al usuario "Tutor" eliminar registros.

REVOKE: QUITA PERMISOS

Elimina un permiso que fue otorgado anteriormente.



Se le quita el permiso que tenía antes.

EJEMPLO

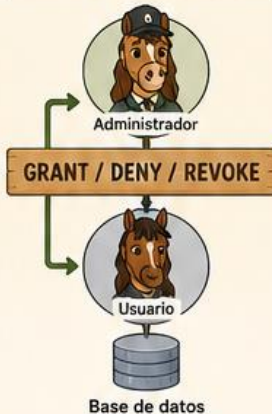
REVOKE INSERT
ON Estudiantes
FROM Tutor;

→ Quita al usuario "Tutor" el permiso de insertar datos.

DIFERENCIAS CLAVE

COMANDO	ACCIÓN
GRANT	Concede permisos.
DENY	Niega permisos (tiene prioridad sobre GRANT).
REVOKE	Elimina permisos otorgados.

¿CÓMO FUNCIONA?



BUENAS PRÁCTICAS

- ✓ Conceder únicamente los permisos necesarios.
- ✓ Usar **GRANT** en lugar de otorgar permisos excesivos.
- ✓ Usar **DENY** solo cuando sea estrictamente necesario.
- ✓ Revisar permisos periódicamente.
- ✓ Usar **REVOKE** para mantener los permisos actualizados.
- ✓ Documentar los cambios realizados.

EJEMPLO EN UN ESTABLO

1. GRANT

El entrenador autoriza al jinete para cuidar al caballo.



El jinete puede alimentarlo y cepillarlo.

2. DENY

El entrenador le prohíbe al jinete vender el caballo.



El jinete NO puede vender el caballo.

3. REVOKE

Luego, el entrenador le quita el permiso de alimentar al caballo.



El jinete ya NO puede alimentarlo.

RECUERDA

El control de acceso adecuado protege la información y evita acciones no autorizadas en la base de datos.

EJEMPLO COMPLETO EN SQL

```
-- Otorgar permiso
GRANT SELECT, INSERT ON Estudiantes TO Tutor;

-- Negar permiso para eliminar
DENY DELETE ON Estudiantes TO Tutor;

-- Quitar permiso de insertar
REVOKE INSERT ON Estudiantes FROM Tutor;
```

BENEFICIOS DEL CONTROL DE ACCESO



Protege los datos sensibles.



Evita accesos y acciones no autorizadas.



Facilita auditorías y cumplimiento de normas.



Mejora la seguridad y confianza en el sistema.



5. CIFRADO Y PROTECCIÓN DE DATOS

TDE y Always Encrypted

Protegiendo la información como nuestros gatitos protegen su rincón favorito



El cifrado protege los datos para que solo las personas autorizadas puedan leerlos. En SQL Server podemos usar TDE para proteger toda la base de datos y Always Encrypted para proteger datos sensibles específicos.



TDE (Transparent Data Encryption)

Cifra toda la base de datos

Protege los archivos de datos y los respaldos sin afectar las aplicaciones.

¿Qué protege?

- Archivos de datos (.mdf, .ndf)
- Archivos de registro (.ldf)
- Copias de seguridad
- Archivos tempdb



Ventajas

- Transparente para las aplicaciones.
- No requiere modificar consultas.
- Protege los archivos incluso si alguien los roba o tiene acceso al almacenamiento.



Always Encrypted

Cifra columnas específicas

Protege los datos sensibles incluso cuando viajan entre la aplicación y SQL Server.

¿Qué protege?

- Columnas específicas (ej. DNI, correo, teléfono, dirección, etc.)
- Los datos permanecen cifrados en la base de datos.
- Ni el administrador de la base de datos puede ver los datos sin la clave.



Ventajas

- Protege datos altamente sensibles.
- Los datos están cifrados en reposo y en tránsito.
- Solo la aplicación con la clave puede descifrarlos.



¿EN QUÉ SE DIFERENCIAN?



	TDE	Always Encrypted
Alcance	Cifra toda la base de datos.	Cifra columnas específicas.
Protege contra	Acceso no autorizado a archivos y respaldos.	Acceso no autorizado a datos sensibles, incluso por administradores.
Transparencia	Totalmente transparente.	Requiere configuración en la aplicación.
Clave	Cifrado por el servidor.	La clave está fuera del servidor (en la aplicación o Key Vault).
Uso recomendado	Para protección general de la base de datos y respaldos.	Para datos altamente confidenciales como DNI, tarjetas, correos, etc.



MEJORES PRÁCTICAS



Usa contraseñas fuertes y protege muy bien las claves.



Habilita TDE para proteger la base de datos y los respaldos.



Usa Always Encrypted para columnas con información muy sensible.



Realiza copias de seguridad periódicamente.



Monitorea y audita el acceso a los datos.



Así como cuidamos y protegemos a nuestros gatitos, también debemos proteger la información que es valiosa para nuestra universidad.
¡La seguridad es responsabilidad de todos!



6. AUDITORÍA Y MONITOREO DE EVENTOS CON SQL SERVER AUDIT



¡VIGILAMOS PARA PROTEGER!

Así como nuestros perritos nos cuidan y nos dan su lealtad, SQL Server Audit cuida nuestra base de datos registrando cada actividad importantes.

¿QUÉ ES UNA AUDITORÍA?



Es el proceso de registrar todas las actividades importantes que ocurren dentro de SQL Server. Permite saber quién, qué, cuándo y desde dónde se realizaron las acciones.

¿POR QUÉ ES IMPORTANTE?

- ✓ Detecta accesos no autorizados.
- ✓ Facilita investigaciones de incidentes.
- ✓ Cumple normativas de seguridad y auditoría.
- ✓ Mejora el control sobre la base de datos.



¿QUÉ EVENTOS PUEDE REGISTRAR?



INICIO Y CIERRE DE SESIÓN

Registra quién se conectó y cuándo.



CREACIÓN / ELIMINACIÓN DE USUARIOS

Controla cambios en logins y usuarios.



ELIMINACIÓN DE OBJETOS

Registra cuando se eliminan tablas, vistas, etc.



CAMBIOS DE PERMISOS

Audita modificaciones en roles y permisos.



MODIFICACIONES DE DATOS

INSERT, UPDATE, DELETE en tablas.



CONSULTAS REALIZADAS

Registra consultas importantes.



ERRORES DE ACCESO

Intentos fallidos y errores de permisos.

COMPONENTES DE SQL SERVER AUDIT

1. AUDIT

Define dónde se almacenarán los registros de auditoría.

Puede guardarlos en:

- 📁 Archivo (en disco)
- 🖥️ Registro de Windows
- 🔒 Registro de Seguridad

2. AUDIT SPECIFICATION

Indica qué acciones o eventos serán auditados.

Ejemplos de acciones:

- 🐾 SELECT
- 🐾 INSERT
- 🐾 DELETE
- 🐾 UPDATE
- 🐾 ALTER TABLE
- 🐾 CREATE LOGIN
- 🐾 EXECUTE
- 🐾 Y más...



¿CÓMO FUNCIONA?



USUARIO REALIZA UNA ACCIÓN



SQL SERVER REGISTRA EL EVENTO (AUDIT)



EL EVENTO SE GUARDA EN EL DESTINO DEFINIDO (ARCHIVO / LOG)



EL ADMINISTRADOR REvisa Y ANALIZA LOS REGISTROS

EJEMPLO EN LA VIDA REAL

Imagina que tu perrito tiene una cámara que registra todo lo que pasa en casa:

- 🐾 Quién entra
- 🐾 Quién saca cosas
- 🐾 Quién abre la puerta
- 🐾 A qué hora sucedió

Así es SQL Server Audit, ¡tu cámara de seguridad para la base de datos!



BUENAS PRÁCTICAS



Audita solo lo necesario para evitar exceso de registros.



Revisa los registros periódicamente.



Protege los archivos de auditoría.



Define una política de retención (cuánto tiempo guardar los registros).



Combina la auditoría con otras medidas de seguridad para mayor protección.

Con SQL Server Audit, mantenemos nuestra base de datos segura, controlada y siempre bajo vigilancia.

¡Porque proteger es amar!

